

COS214 Practical 5

Christian Khaled u25420314

Haytham Phillips u25030907

Nico Sibiya u24667642

Task 1

CampusGuard is an emergency response platform created to manage incidents on a university campus from the initial report to resolution. The system supports the following responsibilities:

Incident Management: Captures incident reports, track situational progression, and regulates state-dependent behavior across reported, active, and resolved phases.

Decoupled Unit Coordination: Facilitates automated, multi-agency field collaboration between Campus Security, Medical Responders, and Facilities personnel during escalating hazards without direct inter-team coupling.

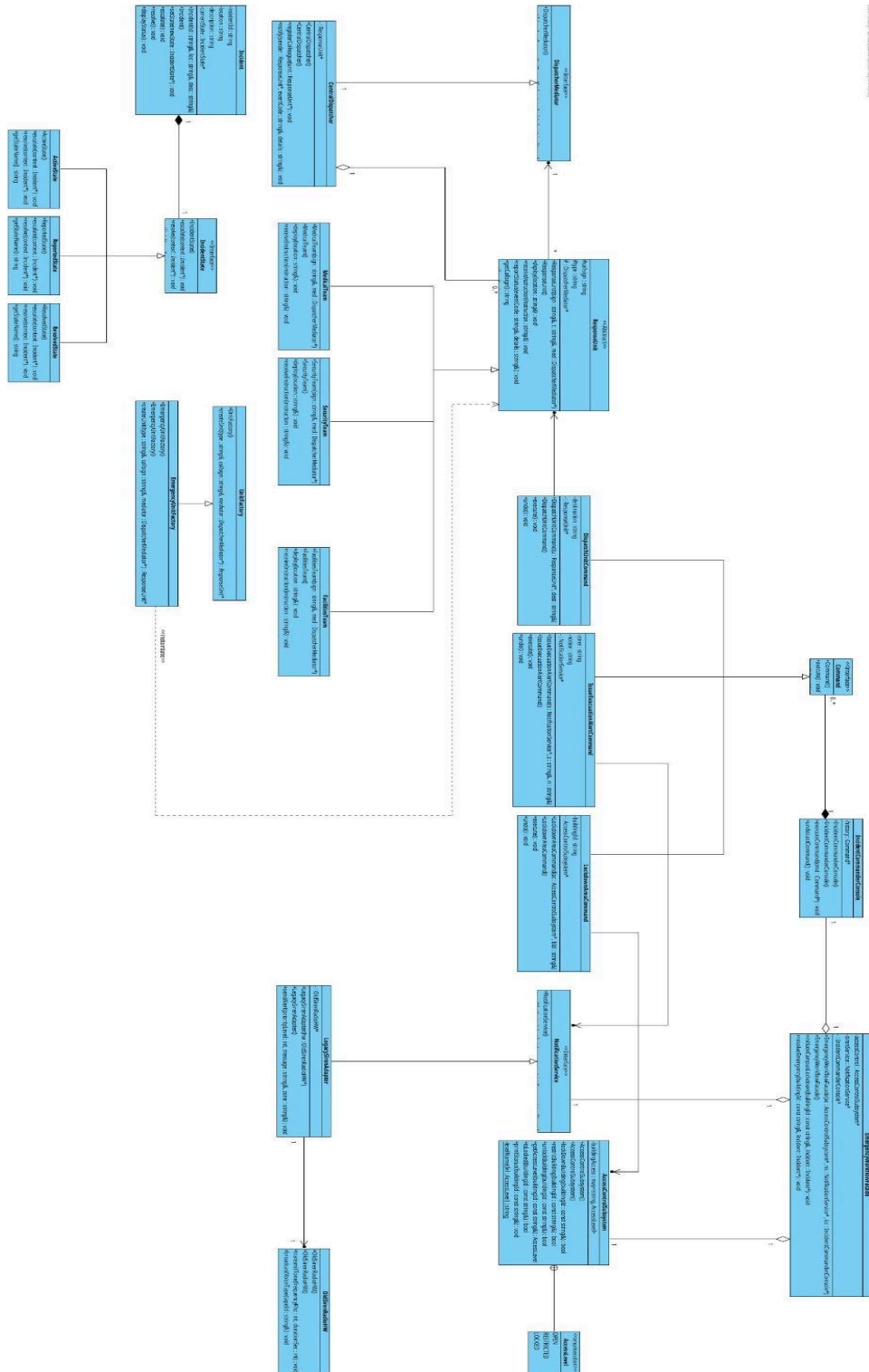
Physical Access and Containment: Governs electronic perimeter security, enabling dynamic locking, unlocking, and isolation of campus structures during crises.

Unified Operator Action Control: Encapsulates tactical operator directives—such as unit dispatching, perimeter lockdown, and public evacuation notices as executable, loggable and undoable objects.

Heterogeneous Infrastructure Integration: Integrates external or legacy infrastructure services (such as legacy radio and tone sirens) through standardized modern messaging contracts.

High-Level Emergency Workflows: Exposes simplified, composite routines that orchestrate multiple critical subsystems simultaneously during high-stress scenarios (e.g., full campus lockdowns).

Final UML Diagram:



Pattern	Participant	Class(es) in CampusGuard
Command	Command	Command
	ConcreteCommand	DispatchUnitCommand
	ConcreteCommand	LockdownAreaCommand
	ConcreteCommand	IssueEvacuationAlertCommand
	Invoker	IncidentCommanderConsole
	Receiver	ResonseUnit, AccessControlSubsystem, NotificationService
Mediator	Mediator	DispatcherMediator
	ConcreteMediator	CentralDispatcher
	Colleague	ResponseUnit
	ConcreteColleague	SecurityTeam
	ConcreteColleague	MedicalTeam
	ConcreteColleague	FacilitiesTeam
Adapter	Target	NotificationService
	Adaptee	OldSirenRadioHW
	Adapter	LegacySirenAdapter
Facade	Facade	EmergencyWorkflowFacade
	Subsystems	AccessControlSubsystem, NotificationService, IncidentCommanderConsole
State (Chosen Pattern)	State	IncidentState
	ConcreteState	ReportedState
	ConcreteState	ActiveState
	ConcreteState	ResolvedState
	Context	Incident
Factory Method (Chosen Pattern)	Creator	UnitFactory
	ConcreteCreator	EmergencyUnitFactory

	Product	ResponseUnit
	ConcreteProduct	SecurityTeam, MedicalTeam, FacilitiesTeam

Justification of Additional Patterns:

State Pattern (Behavioral)

Design Problem Solved: Emergency incidents progress through distinct operational stages (Reported -> Active -> Resolved). Certain operations are strictly illegal during specific phases (e.g. closing an unconfirmed active hazard or deploying emergency lockdowns for an incident that has already been resolved and archived).

A naive implementation relies on complex conditional checks (if (incident->status=="RESOLVED")) scattered across commands, consoles, and facade workflows. The State pattern encapsulates phase-specific rules within distinct state classes, adhering to the Open/Closed Principle and satisfying the non-negotiable requirement to handle invalid runtime operations gracefully without system termination or bloated conditional logic. When an incident escalates, its internal state transitions to ActiveState, permitting lockdown commands to execute. If an operator attempts to issue commands against an incident in ResolvedState, the state intercepts the call, outputs an explanatory error log, and aborts execution cleanly.

Factory Method Pattern (Creational)

Design Problem Solved: The coordination engine requires instantiating specialized response units (SecurityTeam, MedicalTeam, FacilitiesTeam) configured with proper callsigns and automatically attached to the central communication mediator.

Scattering concrete new SecurityTeam(...) instantiations across client workflows or command constructors tightly couples the client layer to concrete classes and risks forgetting to register the unit with the mediator. The Factory Method centralizes responder instantiation and automates registration with CentralDispatcher, ensuring memory ownership is clear and construction errors are caught in one place.

During scenario initialization, client workflows invoke EmergencyUnitFactory::createUnit(), which creates the requested subclass, automatically registers the unit with CentralDispatcher, and returns a polymorphic pointer ready for deployment.

Associations, Inheritance, Multiplicity and Ownership

Polymorphic Destructors: All polymorphic base classes (Command, DispatcherMediator, ResponseUnit, NotificationService, IncidentState, UnitFactory) declare virtual ~[Class] () {} where [Class] is the name of the base class to guarantee safe memory clean up.

IncidentCommanderConsole and Command (Aggregation)

1 IncidentCommanderConsole owns 0 to many Commands. The console takes heap ownership of executed commands. Its destructor iterates through its command history vector and explicitly deletes each Command* to prevent memory leaks.

CentralDispatcher and ResponseUnit (Composition)

1 Dispatcher maintains references to 0 to * Colleagues. The mediator stores non-owning, raw observation pointers (ResponseUnit*). Colleagues are created by EmergencyUnitFactory and owned by the top-level application runtime, preventing circular destruction of dependencies between colleague and mediator.

Incident and IncidentState (Composition)

1 incident owns exactly 1 active IncidentState. Incident owns its state pointer. When setState() is called during a transition, it deallocates the existing state before assigning the new state, and its destructor deallocates the final active state cleanly.

EmergencyWorkflowFacade and Subsystems (Aggregation)

1 Facade references 1 AccessControlSubsystem, 1 NotificationService and 1 IncidentCommanderConsole. The Facade holds non-owning pointers to subsystems passed via dependency injection. Subsystems exist independently and their lifecycles are managed by the parent runtime environment, satisfying the requirement that subsystems remain independently accessible.

LegacySirenAdapter and OldSirenRadioHW (Composition)

1 Adapter references 1 Adaptee instance. Non-owning pointer passed into constructor, allowing the legacy hardware to be shared or configured separately without lifecycle entanglement.

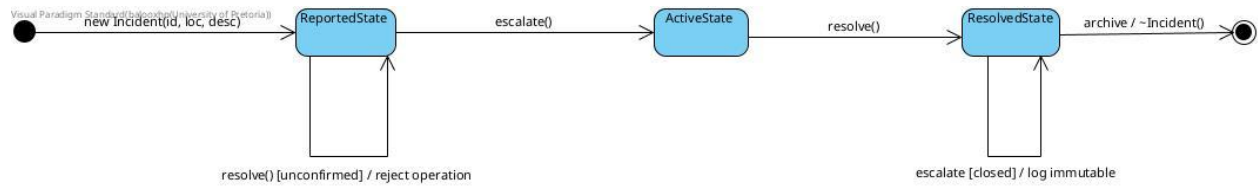
How the 6 patterns interact together:

An event triggers or limits behaviour in other patterns.

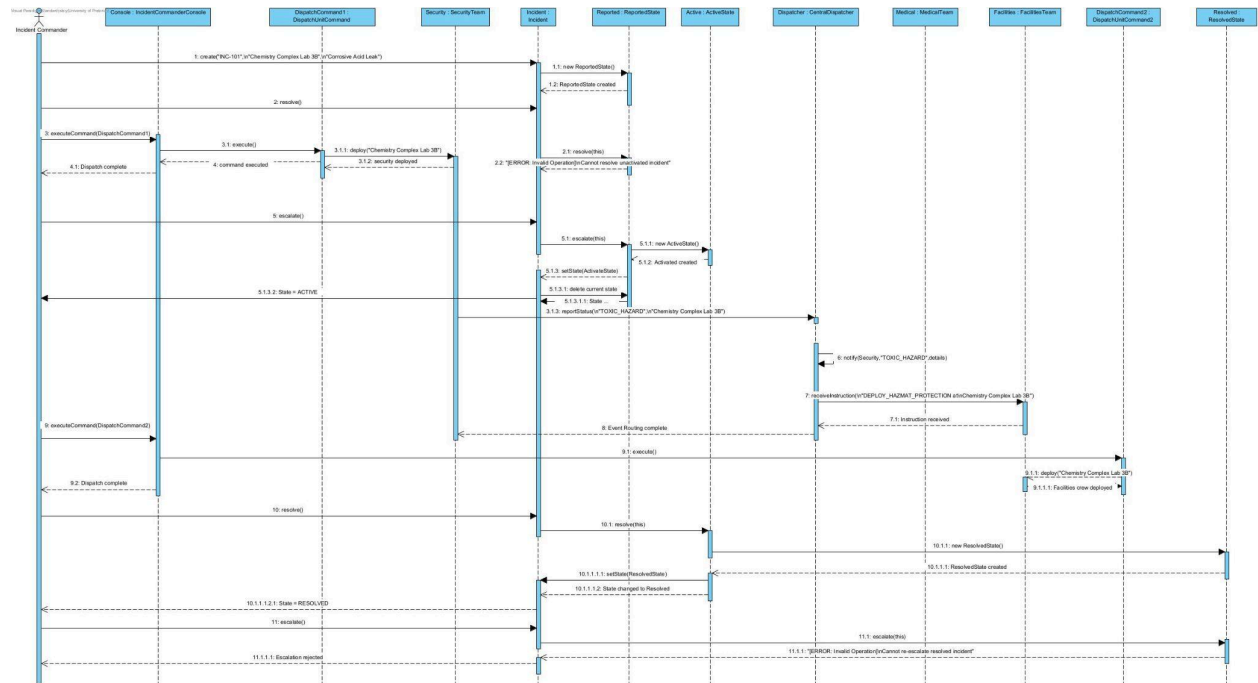
The EmergencyUnitFactory creates the SecurityTeam, MedicalTeam and FacilitiesTeam. They are then all linked to the CentralDispatcher. When a new crisis is logged as an Incident, it starts in ReportedState. Command works with a colleague to execute a DispatchUnitCommand. This triggers the SecurityTeam receiver to move into the specific zone. The Mediator allows a SecurityTeam to call reportStatus() if it detects toxic gas for example. Instead of contacting other units directly it notifies the CentralDispatcher, which gives the MedicalTeam hazmat gear and tells the FacilitiesTeam to stop building ventilation in this case. This now needs to be escalated to a High-Level Lockdown. The EmergencyWorkflowFacade calls initiateCampusLockdown in one sequence: Incident moves to ActiveState, submits a LockdownAreaCommand to the AccessControlSubsystem, send an evacuation alert to the NotificationService which the LegacySirenAdapter converts into a format playable by the OldSirenRadioHW. If an operator tries to close an incident while hazards are not controlled the IncidentState intercepts this action and logs an error, preventing the invalid operation. Once the hazard clears, the Facade transitions Incident into ResolvedState.

Task 4 : UML Diagram portfolio

State Diagram



Chemical Spill Sequence Diagram



Hostile Intruder and Campus LockDown

[illegible]

```

(gdb) next
7      AccessControlSubsystem temp;
(gdb) next
8      temp.lockDownBuilding("TestBuilding");
(gdb) print &temp
$1 = (AccessControlSubsystem *) 0x7ffffffdbf0
(gdb) finish
Run till exit from #0  makeTemp () at main.cpp:8
[AccessControl] TestBuilding -> LOCKED. Electronic badges removed.
0x00005555555589fb in main () at main.cpp:13
13      AccessControlSubsystem* ac = makeTemp();
Value returned is $2 = (AccessControlSubsystem *) 0x0
(gdb) print ac
$3 = (AccessControlSubsystem *) 0x0
(gdb) print *ac
Cannot access memory at address 0x0
(gdb) continue
Continuing.

```

```

Program received signal SIGSEGV, Segmentation fault.
0x0000555555557592 in std::_Rb_tree<std::__cxx11::basic_string<char, std::char_traits<char>,
std::allocator<char> >, std::pair<std::__cxx11::basic_string<char, std::char_traits<char>,
std::allocator<char> > const, AccessControlSubsystem::AccessLevel>,
std::_Select1st<std::pair<std::__cxx11::basic_string<char, std::char_traits<char>,
std::allocator<char> > const, AccessControlSubsystem::AccessLevel> >,
std::less<std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> > >,
std::allocator<std::pair<std::__cxx11::basic_string<char, std::char_traits<char>,
std::allocator<char> > const, AccessControlSubsystem::AccessLevel> > >::_M_begin (this=0x8)
at /usr/include/c++/11/bits/stl_tree.h:745
745      (this->_M_impl._M_header._M_parent);
(gdb) bt
#0  0x0000555555557592 in std::_Rb_tree<std::__cxx11::basic_string<char,
std::char_traits<char>, std::allocator<char> >, std::pair<std::__cxx11::basic_string<char,
std::char_traits<char>, std::allocator<char> > const, AccessControlSubsystem::AccessLevel>,
std::_Select1st<std::pair<std::__cxx11::basic_string<char, std::char_traits<char>,
std::allocator<char> > const, AccessControlSubsystem::AccessLevel> >,
std::less<std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> > >,
std::allocator<std::pair<std::__cxx11::basic_string<char, std::char_traits<char>,
std::allocator<char> > const, AccessControlSubsystem::AccessLevel> > >::_M_begin (this=0x8)
at /usr/include/c++/11/bits/stl_tree.h:745
#1  0x0000555555557173 in std::_Rb_tree<std::__cxx11::basic_string<char,
std::char_traits<char>, std::allocator<char> >, std::pair<std::__cxx11::basic_string<char,
std::char_traits<char>, std::allocator<char> > const, AccessControlSubsystem::AccessLevel>,
std::_Select1st<std::pair<std::__cxx11::basic_string<char, std::char_traits<char>,

```



```

std::allocator<char> > const, AccessControlSubsystem::AccessLevel> >,
std::less<std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> > >,
std::allocator<std::pair<std::__cxx11::basic_string<char, std::char_traits<char>,
std::allocator<char> > const, AccessControlSubsystem::AccessLevel> > >::find (this=0x8,
__k="TestBuilding")
  at /usr/include/c++/11/bits/stl_tree.h:2536
#2 0x00005555555556ee3 in std::map<std::__cxx11::basic_string<char, std::char_traits<char>,
std::allocator<char> >, AccessControlSubsystem::AccessLevel,
std::less<std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> > >,
std::allocator<std::pair<std::__cxx11::basic_string<char, std::char_traits<char>,
std::allocator<char> > const, AccessControlSubsystem::AccessLevel> > >::find (this=0x8,
__x="TestBuilding") at /usr/include/c++/11/bits/stl_map.h:1195
#3 0x00005555555556684 in AccessControlSubsystem::getAccessLevel (this=0x0,
buildingId="TestBuilding") at AccessControlSubsystem.cpp:28
#4 0x00005555555556c24 in AccessControlSubsystem::printStatus (this=0x0,
buildingId="TestBuilding") at AccessControlSubsystem.cpp:91
#5 0x00005555555558a38 in main () at main.cpp:14
(gdb) quit
A debugging session is active.

```

Inferior 1 [process 97340] will be killed.

Quit anyway? (y or n) y

Explanation:

The bug we see here is that makeTemp() creates AccessControlSubsystem temp as a local object on the stack and returns &temp. As soon as this function is returned temp will go out of scope and is destroyed. The pointer that is handed back to main() no longer references a live object and we have a dangling pointer. When we look at the backtrace it helps us trace the crash exactly back to its source. At Frame 3 getAccessLevel tries to call buildingAccess.find on a map member but there is no object address at 0x0. The next frame we see the method main() called on the previously mentioned dangling pointer and lastly frame 5 shows us the original call site. To fix this makeTemp() should not hand out the address of a local variable. The caller needs an object which will extend further than the function call. We must allocate on the heap and make ownership explicit.

Valgrind Evidence:

```

g++ -std=c++11 -Wall -Wextra -g -MMD -MP -c AccessControlSubsystem.cpp -o
AccessControlSubsystem.o
g++ -std=c++11 -Wall -Wextra -g -MMD -MP -c ActiveState.cpp -o ActiveState.o
ActiveState.cpp: In member function 'virtual void ActiveState::escalate(Incident*)':
ActiveState.cpp:8:38: warning: unused parameter 'context' [-Wunused-parameter]
   8 | void ActiveState::escalate(Incident* context) {
     |                               ~~~~~^~~~~~

```

```

g++ -std=c++11 -Wall -Wextra -g -MMD -MP -c CentralDispatcher.cpp -o CentralDispatcher.o
g++ -std=c++11 -Wall -Wextra -g -MMD -MP -c DispatchUnitCommand.cpp -o
DispatchUnitCommand.o
g++ -std=c++11 -Wall -Wextra -g -MMD -MP -c DispatcherMediator.cpp -o DispatcherMediator.o
g++ -std=c++11 -Wall -Wextra -g -MMD -MP -c EmergencyUnitFactory.cpp -o
EmergencyUnitFactory.o
g++ -std=c++11 -Wall -Wextra -g -MMD -MP -c EmergencyWorkflowFacade.cpp -o
EmergencyWorkflowFacade.o
g++ -std=c++11 -Wall -Wextra -g -MMD -MP -c FacilitiesTeam.cpp -o FacilitiesTeam.o
g++ -std=c++11 -Wall -Wextra -g -MMD -MP -c Incident.cpp -o Incident.o
g++ -std=c++11 -Wall -Wextra -g -MMD -MP -c IncidentCommanderConsole.cpp -o
IncidentCommanderConsole.o
g++ -std=c++11 -Wall -Wextra -g -MMD -MP -c IssueEvacuationAlertCommand.cpp -o
IssueEvacuationAlertCommand.o
g++ -std=c++11 -Wall -Wextra -g -MMD -MP -c LegacySirenAdapter.cpp -o
LegacySirenAdapter.o
g++ -std=c++11 -Wall -Wextra -g -MMD -MP -c LockdownAreaCommand.cpp -o
LockdownAreaCommand.o
g++ -std=c++11 -Wall -Wextra -g -MMD -MP -c MedicalTeam.cpp -o MedicalTeam.o
g++ -std=c++11 -Wall -Wextra -g -MMD -MP -c OldSirenRadioHW.cpp -o OldSirenRadioHW.o
g++ -std=c++11 -Wall -Wextra -g -MMD -MP -c ReportedState.cpp -o ReportedState.o
ReportedState.cpp: In member function 'virtual void ReportedState::resolve(Incident*)':
ReportedState.cpp:13:39: warning: unused parameter 'context' [-Wunused-parameter]
   13 | void ReportedState::resolve(Incident* context) {
       |                               ~~~~~^~~~~~
g++ -std=c++11 -Wall -Wextra -g -MMD -MP -c ResolvedState.cpp -o ResolvedState.o
ResolvedState.cpp: In member function 'virtual void ResolvedState::escalate(Incident*)':
ResolvedState.cpp:7:40: warning: unused parameter 'context' [-Wunused-parameter]
    7 | void ResolvedState::escalate(Incident* context) {
       |                               ~~~~~^~~~~~
ResolvedState.cpp: In member function 'virtual void ResolvedState::resolve(Incident*)':
ResolvedState.cpp:11:39: warning: unused parameter 'context' [-Wunused-parameter]
   11 | void ResolvedState::resolve(Incident* context) {
       |                               ~~~~~^~~~~~
g++ -std=c++11 -Wall -Wextra -g -MMD -MP -c ResponseUnit.cpp -o ResponseUnit.o
g++ -std=c++11 -Wall -Wextra -g -MMD -MP -c SecurityTeam.cpp -o SecurityTeam.o
g++ -std=c++11 -Wall -Wextra -g -MMD -MP -c main.cpp -o main.o
g++ -std=c++11 -Wall -Wextra -g -MMD -MP -o campusguard AccessControlSubsystem.o
ActiveState.o CentralDispatcher.o DispatchUnitCommand.o DispatcherMediator.o
EmergencyUnitFactory.o EmergencyWorkflowFacade.o FacilitiesTeam.o Incident.o
IncidentCommanderConsole.o IssueEvacuationAlertCommand.o LegacySirenAdapter.o
LockdownAreaCommand.o MedicalTeam.o OldSirenRadioHW.o ReportedState.o
ResolvedState.o ResponseUnit.o SecurityTeam.o main.o
valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes ./campusguard

```

==112115== Memcheck, a memory error detector
==112115== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==112115== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info
==112115== Command: ./campusguard
==112115==

CAMPUSGUARD: RUNTIME SYSTEM DEMO

SCENARIO 1: Chemical Spill & Toxic Hazard Escalation
Context: Chemistry Complex Lab 3B | Hazardous Liquid Spill

=====

[Incident #INC-101] Location: Chemistry Complex Lab 3B | Details: Corrosive Acid Leak | State: REPORTED

[Testing Invalid Operation 1: Illegal Early Resolution]
[ERROR: Invalid Operation] Cannot resolve an incident that hasn't been triage-activated.

[Operator Action: Dispatching Security Recon]
[SecurityTeam SEC-Alpha] Deployed to secure Chemistry Complex Lab 3B.

[Incident Verification]
[State Transition] Incident verified and escalating from REPORTED to ACTIVE.
[Incident #INC-101] Location: Chemistry Complex Lab 3B | Details: Corrosive Acid Leak | State: ACTIVE

[Field Event: Security Discovers Toxic Fumes]
[CentralDispatcher] Routing event: 'TOXIC_HAZARD' from SEC-Alpha
[MedicalTeam MED-Bravo] Order received: DEPLOY_HAZMAT_PROTECTION at Chemistry Complex Lab 3B.
[FacilitiesTeam FAC-Charlie] Order received: DEPLOY_HAZMAT_PROTECTION at Chemistry Complex Lab 3B.

[Operator Action: Dispatching Facilities Isolation Team]
[FacilitiesTeam FAC-Charlie] Dispatching structural & utility crew to Chemistry Complex Lab 3B.

[Incident Resolution]
[State Transition] Hazard neutralised. Incident moving from ACTIVE to RESOLVED.
[Incident #INC-101] Location: Chemistry Complex Lab 3B | Details: Corrosive Acid Leak | State: RESOLVED

[Testing Invalid Operation 2: Illegal Escalation of Closed Case]
[ERROR: Invalid Operation] Cannot re-escalate an archived/resolved incident.

SCENARIO 1 COMPLETE

SCENARIO 2: Active Campus Intruder & High-Level Emergency Lockdown

Context: Engineering Tower B | Unauthorized Armed Breach

Patterns: Facade, Adapter, Command, State, Mediator

=====

[Incident #INC-202] Location: Engineering Tower B | Details: Armed Intruder Detected | State: REPORTED

[FACADE] Initiating lockdown for Engineering Tower B

[AccessControl] Engineering Tower B -> LOCKED. Electronic badges removed.

[LegacySirenAdapter] Adapting modern alert for zone: Engineering Tower B ('LOCKDOWN: SHELTER IS IN PLACE')

[OldSirenRadioHW] Pushing radio horn at 120 kHz for 15 seconds.

[OldSirenRadioHW] Playing pre-recorded magnetic tape ID:

[TAPE_EVACUATE_ZONE_URGENT].

[State Transition] Incident verified and escalating from REPORTED to ACTIVE.

[FACADE] Lockdown workflow complete

[Subsystem Verification] Is Engineering Tower B locked? YES (LOCKED)

[Operator Action: Issue Broad Campus Evacuation Notice]

[LegacySirenAdapter] Adapting modern alert for zone: Engineering Tower B ('IMMEDIATE EVACUATION REQUIRED')

[OldSirenRadioHW] Pushing radio horn at 120 kHz for 15 seconds.

[OldSirenRadioHW] Playing pre-recorded magnetic tape ID:

[TAPE_EVACUATE_ZONE_URGENT].

[Field Event: Security Engages and Corrals Intruder]

[CentralDispatcher] Routing event: 'HOSTILE_BREACH' from SEC-Bravo

[Threat Neutralized: Executing De-escalation Workflow]

[FACADE] Resolving emergency for Engineering Tower B

[State Transition] Hazard neutralised. Incident moving from ACTIVE to RESOLVED.

[AccessControl] Engineering Tower B -> OPEN. Normal access restored

[LegacySirenAdapter] Adapting modern alert for zone: Engineering Tower B ('ALL CLEAR')

[OldSirenRadioHW] Pushing radio horn at 60 kHz for 5 seconds.

[OldSirenRadioHW] Playing pre-recorded magnetic tape ID: [TAPE_CAUTION_GENERAL].

[FACADE] Resolution workflow complete

[Subsystem Verification] Is Engineering Tower B locked? NO (RESTORED)

SCENARIO 2 COMPLETE

ALL DEMONSTRATION SCENARIOS COMPLETED CLEANLY

```
=====
==112115==
==112115== HEAP SUMMARY:
==112115==    in use at exit: 0 bytes in 0 blocks
==112115==   total heap usage: 55 allocs, 55 frees, 75,271 bytes allocated
==112115==
==112115== All heap blocks were freed -- no leaks are possible
==112115==
==112115== For lists of detected and suppressed errors, rerun with: -s
==112115== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Docker:

DockerFile:

```
FROM ubuntu:22.04

RUN apt-get update && \
    apt-get install -y --no-install-recommends g++ make valgrind gdb && \
    rm -rf /var/lib/apt/lists/*

WORKDIR /app
COPY . /app

RUN make clean && make
CMD ["./campusguard"]
```

Docker-compose.yml:

```
services:
  campusguard:
    build: .
    container_name: campusguard
    stdin_open: true
    tty: true
    cap_add:
      - SYS_PTRACE
    security_opt:
```

```
- seccomp:unconfined
```

Instructions:

Clone the repository

git clone https://github.com/HaythamP/COS214_Prac5

And then cd into it.

Build and run the full docker application:

docker compose up --build

Stop and remove the container afterwards:

docker compose down

Run valgrind inside the docker environment:

docker compose run --rm campusguard valgrind --leak-check=full --show-leak-kinds=all
--track-origins=yes ./campusguard

Run gdb inside of docker environment

docker compose run --rm campusguard gdb ./campusguard

Github Link:

https://github.com/HaythamP/COS214_Prac5

Team Contribution:**Member 1:**

Christian Khaled (u25420314)

Facade, Docker, Makefile, GDB and Valgrind investigation and Task 8

Member 2:

Haytham Phillips (u25030907)

Task 1, Mediator and Adapter pattern, main.cpp, GitHub and one additional behavioural diagram.

Member 3:

Nico Sibiya (u24667642)

Command, State and Factory Method and two Sequence Diagrams

As a team we decided to distribute the work into 3 members, after the work was distributed we all started working on our own individual parts committing to our own branches that were created under our name. Once everything was committed to our individual branch, member 2 merged everything to a dev branch to test and see if all the code was working together and

there were no mistakes. Once all the code was running and working it was pushed to main for submission where we made a couple small touches such as editing the README